MAESTROS A UN CLIC

Plan y reporte de pruebas

Plataforma web que conecta clientes con profesionales verificados de servicios del hogar en Medellín

Integrantes

- Juan Pablo Castaño
- Brayan Arenas

Institución Universitaria Pascual Bravo
Tecnología en Desarrollo de Software · Sexto semestre
Ingeniería de Software 2

Docente: Maira Perez

Medellín, Octubre de 2026

1. Objetivo

Verificar de forma automática y repetible que cada historia de usuario cumple sus criterios de aceptación, que las reglas de negocio (agenda, estados de la reserva, permisos) no se rompen al agregar funcionalidades, y que la calidad del código se mantiene.

2. Estrategia

Nivel	Qué se prueba	Herramienta	Dónde
Pruebas de API (integración)	Cada endpoint: caso feliz, validación (422), permisos (401/403), reglas de estado (409) y efectos (registros, notificaciones, eventos, archivos)	Pest (PHPUnit) sobre SQLite en memoria	Backend `tests/Feature`
Pruebas unitarias del frontend	Funciones puras: fechas en hora de Colombia, rangos, horario semanal, esquemas de validación, cliente HTTP, formato	Vitest	Frontend `__tests__`
Pruebas de componentes	Interacción principal de cada componente con datos: lo que se muestra, lo que se envía a la API y cómo se manejan los errores	Vitest + Vue Test Utils (API simulada)	Frontend `__tests__`
Análisis estático	Tipos, estilo, imports entre módulos y colores fuera del sistema de diseño	TypeScript (vue-tsc), ESLint, oxlint, Pint, Prettier y verificador de tokens	Ambos
Seguridad de dependencias	Vulnerabilidades conocidas en las librerías	`composer audit`, `npm audit`	CI
Pruebas manuales	Recorridos completos en el navegador con dos sesiones (cliente y profesional), tiempo real y responsive	Navegador	Antes de fusionar cada HU

Reglas de la estrategia:

- Toda historia nueva incluye sus pruebas en el mismo pull request. Sin pruebas no cumple la definición de terminado.
- Por endpoint, como mínimo: caso feliz, validación y permisos.
- Las pruebas no dependen de servicios externos: la API se simula en el frontend, y en el backend se simulan las notificaciones, los eventos y el almacenamiento (`Notification::fake`, `Event::fake`, `Storage::fake`).
- Las fechas se fijan en las pruebas que dependen de la hora (`Carbon::setTestNow`, `vi.useFakeTimers`), para que den el mismo resultado cualquier día.
- No se permite saltar, enfocar ni borrar pruebas para que la CI pase: el check **Guardas** lo detecta y lo marca en el PR.

3. Integración continua

Cada pull request a `main` ejecuta automáticamente, y GitHub no permite fusionar si algo falla:

Repositorio	Verificaciones obligatorias
Backend	Formato (Pint) y pruebas (Pest) en **PHP 8.2 y 8.5**; Guardas
Frontend	Lockfile, lint (oxlint, ESLint, tokens de diseño), formato, tipos, pruebas y build de producción; Guardas
Ambos (informativo)	Auditoría de dependencias

4. Reporte de resultados

Ejecución del 22 de octubre de 2026 sobre `main`, con las 49 historias terminadas:

Suite	Pruebas	Resultado	Aserciones	Duración
Backend (Pest)	220	✅ 220 aprobadas	992	≈ 23 s
Frontend (Vitest)	158 en 38 archivos	✅ 158 aprobadas	—	≈ 60 s (en Windows)

4.1 Backend: pruebas por archivo

Archivo	Qué cubre
BookingsTest	Crear reservas (horario, bloqueos, cruces, precio), aceptar o rechazar, confirmar, iniciar y finalizar, listados y filtros, resumen, notificaciones
ProfessionalProfileTest	Perfil, especialidades, servicios con duración, portafolio y permisos
AvailabilityTest	Horario semanal, descanso, bloqueos, cruces y evento de agenda
BookingRescheduleTest	Reprogramar y cancelar con sus reglas y notificaciones
ChatTest	Enviar texto e imágenes, chat cerrado, permisos, leídos y eliminar mensajes
ApiErrorsTest	Formato estándar de errores (404, 405, 422, 401, 403)
ProfessionalsTest	Búsqueda pública, filtros, orden, cercanía, comunas aledañas, próxima hora libre y privacidad del perfil
ProfessionalSlotsTest	Cálculo de horas libres
NotificationsTest	Listado, contador y marcar como leídas
EarningsTest	Ingresos por rango y validaciones
BookingHistoryTest	Historial de estados y evento de estado

Archivo	Qué cubre
Auth/*, RegisterClientTest	Registro, inicio y cierre de sesión, y recuperación de contraseña sin revelar qué correos existen
AccountTest	Editar datos y ubicación, foto, cambio de contraseña y eliminación de cuenta con anonimización
VerificationTest, AdminVerificationTest	Cargue y reemplazo de documentos, y revisión del administrador
ReviewsTest	Calificación recíproca y promedio
UserReportsAndModerationTest	Reportes, bloqueo de usuarios, alertas, perfil del cliente, comentarios públicos y "disponible ahora"
FavoritesTest, CatalogTest, BroadcastingTest, ApiDocsTest, DatabaseSeederTest	Favoritos, catálogos, autorización de canales, documentación de la API y datos de demostración

4.2 Frontend: pruebas por área

Área	Archivos
Reservas	BookingResponseActions, ConfirmBookingButton, ServiceProgressCard, RescheduleBookingModal, BookingHistory, BookingList, InProgressPanel
Agenda	SlotPicker, WeeklyScheduleEditor, weekly, dates
Chat y tiempo real	BookingChat, presence
Notificaciones	NotificationBell
Perfil profesional	schemas, ServiceFormModal, EarningsView, ranges
Cuenta	ChangePasswordForm, DeleteAccountModal, ManualLocationForm, schemas, PasswordRecovery, RegisterProfessionalForm
Verificación y administración	AdminVerifications, ReplaceDocumentModal, schemas
Calificaciones y reportes	ReviewBookingModal, ReportUserModal, ProfessionalCard
Base	router (guards), cliente HTTP, formularios, formato, tokens, verificador de colores, favoritos, esquemas de auth

4.3 Pruebas funcionales sobre la aplicación

Además de las suites automatizadas, para la entrega final se ejecutó una batería de **44 pruebas funcionales de caja negra** sobre la aplicación en marcha: 28 positivas y 16 negativas, con Chrome para los flujos de pantalla. Las 44 se aprobaron el 23 de octubre de 2026. También se revisaron 29 pantallas en tres anchos (celular, tablet y escritorio). La matriz, los resultados y los hallazgos están en el informe de análisis funcional, y los scripts, en `_pruebas/`.

5. Cómo ejecutar las pruebas

```
# Backend
php artisan test
vendor/bin/pint --test

# Frontend
npm run lint
npm run type-check
npx vitest run
npm run build
```

6. Pendiente

- **Cobertura de código medida** (porcentaje de líneas): activar `--coverage` en los dos repositorios y publicar el resultado en este reporte.
- **Pruebas de extremo a extremo** de los recorridos principales con un navegador automatizado (Playwright).
- **Prueba de carga** para el requisito RNF-REN-01, sobre el servidor de producción.
- **Auditoría de accesibilidad** (Lighthouse) para RNF-USA-04.
- **Prueba automatizada del cierre por inactividad** (HU050) y de las vistas del administrador (usuarios, reportes y alertas), que hoy se prueban a mano.